

Task Name: React Component Lifecycle Methods

Code:

Post.js

```
class Post {

  constructor(userId, id, title, body) {

    this.userId = userId;
    this.id = id;
    this.title = title;
    this.body = body;
  }

}

export default Post;
```

Post.jsx

```
import React, { Component } from "react";
import Post from "../Post";

class Posts extends Component {

  constructor(props) {

    super(props);

    this.state = {

      posts: []
    };
  }

  loadPosts() {

    fetch("https://jsonplaceholder.typicode.com/posts")

      .then((response) => response.json())

      .then((data) => {
```

```
        const postList = data.map((post) =>

            new Post(
                post.userId,
                post.id,
                post.title,
                post.body
            )
        );

        this.setState({
            posts: postList
        });
    })

    .catch((error) => {

        console.error(error);

    });
}

componentDidMount() {

    this.loadPosts();
}

componentDidCatch(error, info) {

    alert("An error occurred.");

    console.error(error);
    console.error(info);
}

render() {

    return (

        <div>

            <h1>Blog Posts</h1>
```

```

        {
            this.state.posts.map((post) => (
                <div key={post.id}>
                    <h3>{post.title}</h3>
                    <p>{post.body}</p>
                    <hr />
                </div>
            ))
        }
    </div>
);
}
}

export default Posts;

```

App.jsx

```

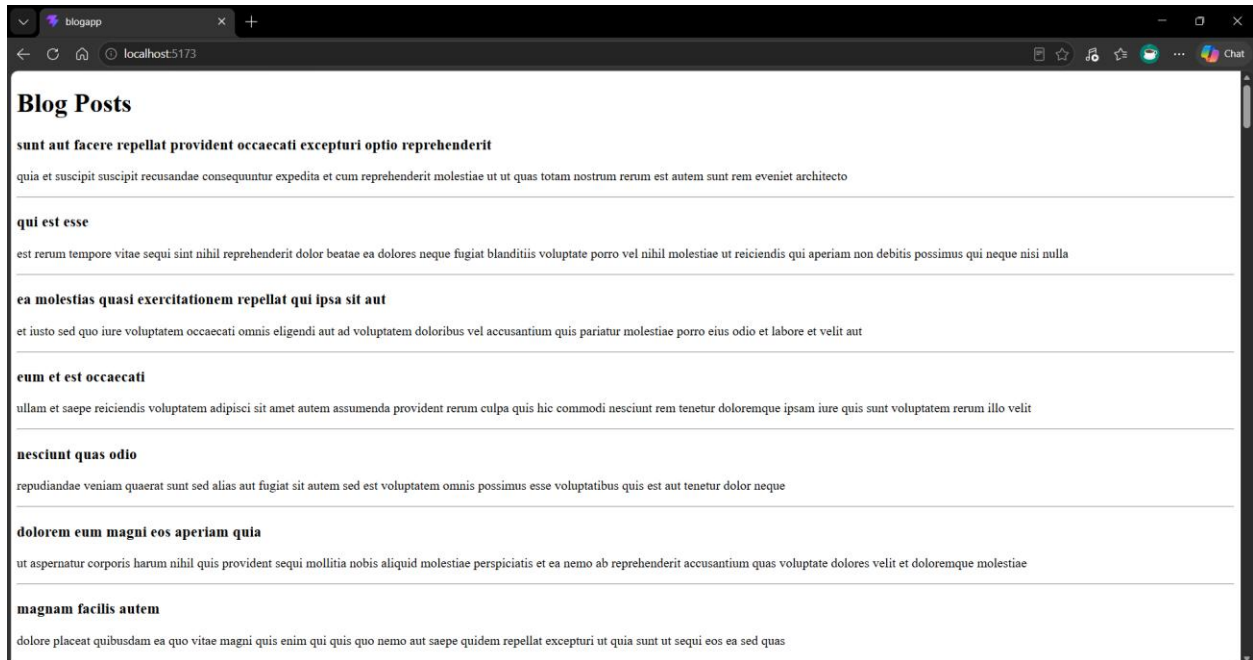
import Posts from "../Posts";

function App() {
    return (
        <div>
            <Posts />
        </div>
    );
}

export default App;

```

Output:



Theory:

What is the Component Lifecycle?

The React Component Lifecycle refers to the different stages that a component goes through during its existence:

1. Mounting
2. Updating
3. Unmounting
4. Error Handling

Lifecycle methods allow developers to perform specific actions during these stages.

Benefits of Lifecycle Methods

- Fetch data from APIs.
- Handle component initialization.
- Manage updates efficiently.
- Handle errors gracefully.
- Improve application reliability.

Important Lifecycle Methods

Lifecycle Method	Purpose
constructor()	Initialize state and properties
componentDidMount()	Executes after the component is rendered
render()	Displays UI elements
componentDidCatch()	Handles component errors
componentWillUnmount()	Cleanup operations before component removal

Lifecycle Sequence:

Constructor()



Render()



componentDidMount()



State Updates



Render()



componentDidCatch() (if any errors occur)